Database Systems Project

Technical Training Center Management System

Prepared by:

Omar Siyam
Hala Imad

1. Introduction and Scenario

A professional technical training center delivers short-term and long-term courses in programming, databases, mobile development, networking, multimedia, business applications, and other technical areas. The center currently relies on scattered spreadsheets and documents, which makes it difficult to maintain consistent information and produce management reports.

The purpose of this project is to analyze the center's requirements and design a normalized relational database in Microsoft SQL Server. The database separates the course catalog from individual course offerings, records departments, instructors, rooms, trainees, enrollments, assessments, and results, and preserves historical enrollment and assessment information after an offering is completed.

2. Business Requirements

The system must maintain training departments and their courses. Every course belongs to one department, while a department may be responsible for several courses. Departments may have a coordinator instructor when applicable.

The system must maintain instructors with personal, contact, professional, employment, department, and status information. An instructor may teach several offerings or may have no current teaching assignment and must still remain visible.

The course catalog stores courses independently from their deliveries. A course can have many offerings over time or no current offering. Each offering records its term, dates, trainee capacity, status, and optional instructor and room assignment.

Trainees register with the center before enrolling. A trainee can exist without an enrollment. Enrollment records the many-to-many relationship between trainees and course offerings and includes the enrollment date and status. A trainee cannot be enrolled twice in the same offering.

Assessments belong to individual course offerings. Each result stores the mark obtained by a trainee. The system should support retrieval of individual marks, averages, highest and lowest marks, course performance, and completion information.

3. Data Requirements

- Department: identifier, name, description, and optional coordinator instructor.
- Instructor: identifier, name, email, phone, specialization, employment date, status, and department.

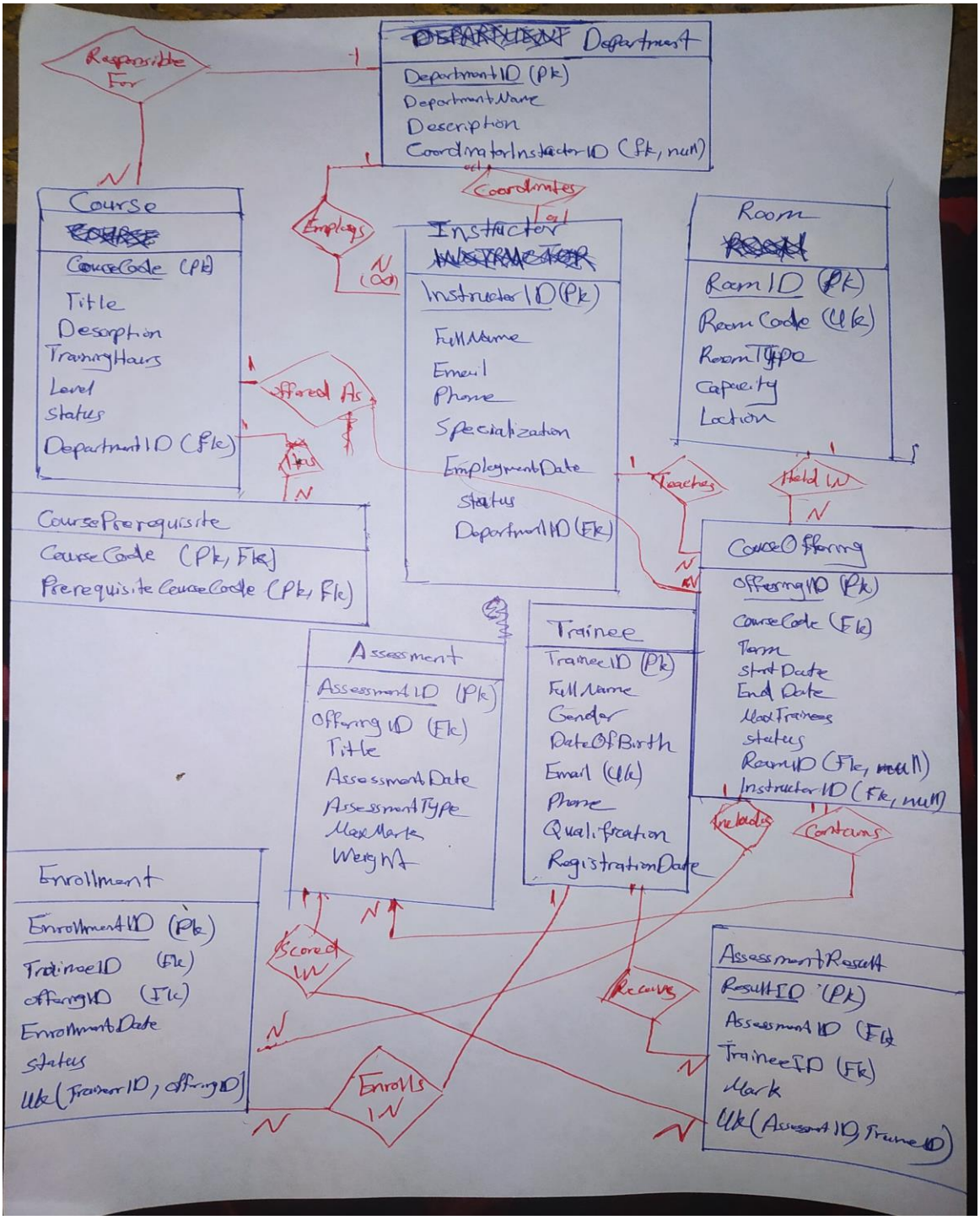
- Course: course code, title, description, training hours, level, status, and responsible department.
- Room: room identifier, room code, type, capacity, and location.
- CourseOffering: offering identifier, course, term, dates, maximum trainees, status, optional instructor, and optional room.
- Trainee: identifier, name, gender, date of birth, email, phone, qualification, and registration date.
- Assessment: identifier, offering, title, type, date, maximum mark, and weight.
- CoursePrerequisite: course code and prerequisite course code.
- Enrollment: identifier, trainee, offering, enrollment date, and status.
- AssessmentResult: result identifier, assessment, trainee, and mark.

4. Business Rules

- Every course belongs to one responsible department, and a department may be responsible for several courses.
- A course may have several offerings over time, and each offering belongs to one course.
- A course may exist in the catalog without having a current offering.
- An instructor may teach several offerings and may currently teach none.
- A trainee may enroll in several offerings, and an offering may contain many trainees.
- A trainee must not enroll more than once in the same offering.
- Every assessment belongs to one course offering, and an offering may contain several assessments.
- A trainee should receive a result only for an offering in which that trainee is enrolled.
- Assessment marks must be non-negative and should not exceed the corresponding assessment maximum.
- Course prerequisites must refer to valid courses, and a course must not be its own prerequisite.
- A room may be used for different offerings at different times.
- Registered trainees, unassigned instructors, and courses without current offerings must remain visible.

5. ER Diagram

The conceptual ER model contains the ten entities used in the final design. CoursePrerequisite resolves the recursive many-to-many prerequisite relationship between courses. Enrollment resolves the many-to-many relationship between trainees and course offerings. The ER diagram must be kept consistent with the relational schema and SQL implementation.



6. Relational Schema

The following schema uses CourseCode as the primary key of Course. AssessmentResult uses ResultID as its primary key and stores AssessmentID, TraineeID, and Mark. Optional relationships are represented by nullable foreign keys in CourseOffering and Department.

Department

Attribute	Key / Nullability	Description
DepartmentID	PK, IDENTITY	Department identifier
DepartmentName	NOT NULL, UNIQUE	Department name
Description	NULL	Descriptive information
CoordinatorInstructorID	NULL, FK	Optional coordinator

Instructor

Attribute	Key / Nullability	Description
InstructorID	PK, IDENTITY	Instructor identifier
FullName	NOT NULL	Instructor name
Email	NOT NULL, UNIQUE	Contact email
Phone	NULL	Contact phone
Specialization	NOT NULL	Professional specialization
EmploymentDate	NOT NULL	Employment date
Status	NOT NULL, DEFAULT, CHECK	Active or Inactive
DepartmentID	NOT NULL, FK	Instructor department

Course

Attribute	Key / Nullability	Description
CourseCode	PK	Course catalog code
Title	NOT NULL	Course title
Description	NULL	Course description
TrainingHours	NOT NULL, CHECK > 0	Training duration
Level	NOT NULL, CHECK	Beginner, Intermediate, or Advanced
Status	NOT NULL, DEFAULT, CHECK	Active or Inactive
DepartmentID	NOT NULL, FK	Responsible department

Room

Attribute	Key / Nullability	Description
RoomID	PK, IDENTITY	Room identifier
RoomCode	NOT NULL, UNIQUE	Room code
RoomType	NOT NULL, CHECK	Allowed room type
Capacity	NOT NULL, CHECK > 0	Room capacity
Location	NOT NULL	Room location

CourseOffering

Attribute	Key / Nullability	Description
OfferingID	PK, IDENTITY	Offering identifier
CourseCode	NOT NULL, FK	Catalog course
Term	NOT NULL	Term or training period
StartDate	NOT NULL	Starting date
EndDate	NOT NULL, CHECK	Ending date
MaxTrainees	NOT NULL, CHECK > 0	Offering capacity
Status	NOT NULL, CHECK	Planned, Open, In Progress, Completed, or Cancelled
InstructorID	NULL, FK	Optional assigned instructor
RoomID	NULL, FK	Optional assigned room

Trainee

Attribute	Key / Nullability	Description
TraineeID	PK, IDENTITY	Trainee identifier
FullName	NOT NULL	Trainee name
Gender	NOT NULL, CHECK	Male or Female
DateOfBirth	NOT NULL	Date of birth
Email	NOT NULL, UNIQUE	Unique email
Phone	NULL	Contact phone
Qualification	NOT NULL	Educational qualification
RegistrationDate	NOT NULL, DEFAULT	Registration date

Assessment

Attribute	Key / Nullability	Description
AssessmentID	PK, IDENTITY	Assessment identifier
OfferingID	NOT NULL, FK	Course offering
Title	NOT NULL	Assessment title
AssessmentType	NOT NULL	Type of assessment
AssessmentDate	NOT NULL	Assessment date
MaxMark	NOT NULL, CHECK > 0	Maximum mark
Weight	NOT NULL, CHECK 0–100	Assessment weight

CoursePrerequisite

Attribute	Key / Nullability	Description
CourseCode	PK/FK	Course requiring prerequisite
PrerequisiteCourseCode	PK/FK	Required course
(CourseCode, PrerequisiteCourseCode)	Composite PK, CHECK not self	Associative recursive key

Enrollment

Attribute	Key / Nullability	Description
EnrollmentID	PK, IDENTITY	Enrollment identifier
TraineeID	NOT NULL, FK	Enrolled trainee
OfferingID	NOT NULL, FK	Selected offering
EnrollmentDate	NOT NULL	Enrollment date
Status	NOT NULL, CHECK	Registered, Withdrawn, Completed, or Failed
(TraineeID, OfferingID)	UNIQUE	Prevents duplicate enrollment

AssessmentResult

Attribute	Key / Nullability	Description
ResultID	PK, IDENTITY	Result identifier
AssessmentID	NOT NULL, FK	Assessment
TraineeID	NOT NULL, FK	Trainee
Mark	NOT NULL, CHECK >= 0	Obtained mark
(AssessmentID, TraineeID)	UNIQUE	One result per trainee per assessment

7. Normalization

7.1 First Normal Form (1NF)

The relations are in 1NF because each column stores a single value and there are no repeating groups or multi-valued fields. For example, a trainee has one email value in Trainee, while separate enrollments are stored as separate rows in Enrollment rather than as a list inside the trainee record.

7.2 Second Normal Form (2NF)

The design avoids partial dependencies. Enrollment represents a relationship and its descriptive attributes, such as EnrollmentDate and Status, apply to the enrollment record. The unique pair (TraineeID, OfferingID) identifies the relationship, and neither enrollment attribute depends on only one side of the relationship.

7.3 Third Normal Form (3NF)

The final design is in 3NF because non-key attributes depend on the key, the whole key, and nothing but the key. Course details are stored once in Course and are referenced by CourseOffering. Trainee details are stored once in Trainee and are referenced by Enrollment. AssessmentResult stores the mark for an assessment and trainee without duplicating assessment title, maximum mark, or trainee contact information.

CourseOffering and Enrollment are important examples. If course title, term, trainee name, and enrollment status were stored repeatedly in one large table, updates would create duplication and inconsistency. Separating these relations removes that duplication and supports courses with many offerings and offerings with many trainees.

8. Database Design Decisions

- CourseCode is used as the primary key because it is the stable identifier used in the catalog and prerequisite relationships.
- CourseOffering is separated from Course because one catalog course can be delivered in different terms with different dates, instructors, rooms, statuses, and capacities.
- Enrollment is a separate relation because trainees and offerings have a many-to-many relationship.
- CoursePrerequisite is a separate associative relation because a course can have many prerequisites and can be a prerequisite for many other courses.
- Assessment is related to CourseOffering because different deliveries of the same course may use different assessment structures.
- AssessmentResult stores the individual mark for a trainee and an assessment; ResultID gives each result a simple identifier.
- InstructorID and RoomID in CourseOffering are nullable because scheduling or instructor assignment may happen after an offering is created.
- UNIQUE constraints protect candidate keys such as email, room code, and the trainee-offering pair.
- CHECK constraints protect simple domain and range rules such as status values, positive capacities, valid levels, and non-negative marks.
- The design avoids unnecessary duplication by storing each fact with the entity or relationship to which it belongs.

9. Database Implementation – DDL

The database was implemented in Microsoft SQL Server. The following script creates the database and all ten tables in dependency order. Primary keys, foreign keys, UNIQUE constraints, NOT NULL constraints, CHECK constraints, and DEFAULT constraints are included. The Department coordinator foreign key is added with ALTER TABLE after Instructor exists.

```
CREATE DATABASE TechnicalTrainingCenter;
GO
USE TechnicalTrainingCenter;
GO

CREATE TABLE Department (
    DepartmentID INT IDENTITY(1,1) PRIMARY KEY,
    DepartmentName NVARCHAR(100) NOT NULL UNIQUE,
    Description NVARCHAR(255) NULL,
    CoordinatorInstructorID INT NULL
);

CREATE TABLE Instructor (
    InstructorID INT IDENTITY(1,1) PRIMARY KEY,
    FullName NVARCHAR(120) NOT NULL,
    Email NVARCHAR(150) NOT NULL UNIQUE,
    Phone NVARCHAR(30) NULL,
    Specialization NVARCHAR(120) NOT NULL,
    EmploymentDate DATE NOT NULL,
    Status NVARCHAR(20) NOT NULL DEFAULT 'Active',
    DepartmentID INT NOT NULL,
    CONSTRAINT CK_Instructor_Status CHECK (Status IN ('Active', 'Inactive')),
    CONSTRAINT FK_Instructor_Department FOREIGN KEY (DepartmentID)
        REFERENCES Department(DepartmentID)
);

CREATE TABLE Course (
    CourseCode VARCHAR(10) PRIMARY KEY,
    Title NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    TrainingHours INT NOT NULL,
    Level NVARCHAR(20) NOT NULL,
    Status NVARCHAR(20) NOT NULL DEFAULT 'Active',
    DepartmentID INT NOT NULL,
    CONSTRAINT CK_Course_TrainingHours CHECK (TrainingHours > 0),
    CONSTRAINT CK_Course_Level CHECK (Level IN ('Beginner', 'Intermediate', 'Advanced')),
    CONSTRAINT CK_Course_Status CHECK (Status IN ('Active', 'Inactive')),
    CONSTRAINT FK_Course_Department FOREIGN KEY (DepartmentID)
        REFERENCES Department(DepartmentID)
);

CREATE TABLE Room (
    RoomID INT IDENTITY(1,1) PRIMARY KEY,
    RoomCode VARCHAR(20) NOT NULL UNIQUE,
    RoomType NVARCHAR(40) NOT NULL,
    Capacity INT NOT NULL,
    Location NVARCHAR(120) NOT NULL,
    CONSTRAINT CK_Room_Capacity CHECK (Capacity > 0),
    CONSTRAINT CK_Room_Type CHECK (RoomType IN ('Computer Laboratory', 'Regular
Classroom', 'Multimedia Laboratory'))
```

```
);

CREATE TABLE CourseOffering (
    OfferingID INT IDENTITY(1,1) PRIMARY KEY,
    CourseCode VARCHAR(10) NOT NULL,
    Term NVARCHAR(50) NOT NULL,
    StartDate DATE NOT NULL,
    EndDate DATE NOT NULL,
    MaxTrainees INT NOT NULL,
    Status NVARCHAR(25) NOT NULL,
    InstructorID INT NULL,
    RoomID INT NULL,
    CONSTRAINT CK_Offering_Dates CHECK (EndDate >= StartDate),
    CONSTRAINT CK_Offering_MaxTrainees CHECK (MaxTrainees > 0),
    CONSTRAINT CK_Offering_Status CHECK (Status IN ('Planned', 'Open', 'In Progress',
'Completed', 'Cancelled')),
    CONSTRAINT FK_Offering_Course FOREIGN KEY (CourseCode) REFERENCES Course(CourseCode),
    CONSTRAINT FK_Offering_Instructor FOREIGN KEY (InstructorID) REFERENCES
Instructor(InstructorID),
    CONSTRAINT FK_Offering_Room FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
);

CREATE TABLE Trainee (
    TraineeID INT IDENTITY(1,1) PRIMARY KEY,
    FullName NVARCHAR(120) NOT NULL,
    Gender NVARCHAR(20) NOT NULL,
    DateOfBirth DATE NOT NULL,
    Email NVARCHAR(150) NOT NULL UNIQUE,
    Phone NVARCHAR(30) NULL,
    Qualification NVARCHAR(150) NOT NULL,
    RegistrationDate DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),
    CONSTRAINT CK_Trainee_Gender CHECK (Gender IN ('Male', 'Female'))
);

CREATE TABLE Assessment (
    AssessmentID INT IDENTITY(1,1) PRIMARY KEY,
    OfferingID INT NOT NULL,
    Title NVARCHAR(120) NOT NULL,
    AssessmentType NVARCHAR(40) NOT NULL,
    AssessmentDate DATE NOT NULL,
    MaxMark DECIMAL(5,2) NOT NULL,
    Weight DECIMAL(5,2) NOT NULL,
    CONSTRAINT CK_Assessment_MaxMark CHECK (MaxMark > 0),
    CONSTRAINT CK_Assessment_Weight CHECK (Weight >= 0 AND Weight <= 100),
    CONSTRAINT FK_Assessment_Offering FOREIGN KEY (OfferingID)
        REFERENCES CourseOffering(OfferingID)
);

CREATE TABLE CoursePrerequisite (
    CourseCode VARCHAR(10) NOT NULL,
    PrerequisiteCourseCode VARCHAR(10) NOT NULL,
    CONSTRAINT PK_CoursePrerequisite PRIMARY KEY (CourseCode, PrerequisiteCourseCode),
    CONSTRAINT CK_CoursePrerequisite_NotSelf CHECK (CourseCode <> PrerequisiteCourseCode),
    CONSTRAINT FK_CoursePrerequisite_Course FOREIGN KEY (CourseCode) REFERENCES
Course(CourseCode),
    CONSTRAINT FK_CoursePrerequisite_Prerequisite FOREIGN KEY (PrerequisiteCourseCode)
REFERENCES Course(CourseCode)
);

CREATE TABLE Enrollment (
    EnrollmentID INT IDENTITY(1,1) PRIMARY KEY,
```

```
    TraineeID INT NOT NULL,
    OfferingID INT NOT NULL,
    EnrollmentDate DATE NOT NULL,
    Status NVARCHAR(20) NOT NULL,
    CONSTRAINT UQ_Enrollment_Trainee_Offering UNIQUE (TraineeID, OfferingID),
    CONSTRAINT CK_Enrollment_Status CHECK (Status IN ('Registered', 'Withdrawn',
'Completed', 'Failed')),
    CONSTRAINT FK_Enrollment_Trainee FOREIGN KEY (TraineeID) REFERENCES
Trainee(TraineeID),
    CONSTRAINT FK_Enrollment_Offering FOREIGN KEY (OfferingID) REFERENCES
CourseOffering(OfferingID)
);

CREATE TABLE AssessmentResult (
    ResultID INT IDENTITY(1,1) PRIMARY KEY,
    AssessmentID INT NOT NULL,
    TraineeID INT NOT NULL,
    Mark DECIMAL(5,2) NOT NULL,
    CONSTRAINT UQ_AssessmentResult_Assessment_Trainee UNIQUE (AssessmentID, TraineeID),
    CONSTRAINT CK_AssessmentResult_Mark CHECK (Mark >= 0),
    CONSTRAINT FK_AssessmentResult_Assessment FOREIGN KEY (AssessmentID) REFERENCES
Assessment(AssessmentID),
    CONSTRAINT FK_AssessmentResult_Trainee FOREIGN KEY (TraineeID) REFERENCES
Trainee(TraineeID)
);

ALTER TABLE Department
ADD CONSTRAINT FK_Department_Coordinator
FOREIGN KEY (CoordinatorInstructorID) REFERENCES Instructor(InstructorID);
GO
```

This is the link to the SSMS program for the DDL file.

https://drive.google.com/file/d/1Ek_GEG9bKRjPdeVxhD5tIuKOTmN4BMPp/view?usp=sharing

10. Test Data and DML

The sample data is intended to test normal operations and the required query concepts. It includes several departments and courses, multiple offerings for DB201, a course with no current offering, instructors with different assignment situations, trainees with different enrollment situations, varied statuses, prerequisite relationships, multiple assessments, and varied results.

```
USE TechnicalTrainingCenter;
GO

INSERT INTO Department (DepartmentName, Description) VALUES
('Programming and Databases', 'Programming languages, database systems, and software
development.'),
('Networking', 'Networking fundamentals, administration, and infrastructure.'),
('Multimedia', 'Digital design, media production, and multimedia applications.'),
('Business Applications', 'Office productivity and business information systems.');

INSERT INTO Instructor (FullName, Email, Phone, Specialization, EmploymentDate, Status,
DepartmentID) VALUES
('Dr. Lina Haddad', 'lina.haddad@ttc.edu', '0597001001', 'Database Systems', '2019-09-01',
'Active', 1),
('Eng. Samer Nassar', 'samer.nassar@ttc.edu', '0597001002', 'Web and Mobile Development',
'2020-02-15', 'Active', 1),
('Maya Saleh', 'maya.saleh@ttc.edu', '0597001003', 'Computer Networks', '2018-06-10',
'Active', 2),
('Omar Khalil', 'omar.khalil@ttc.edu', '0597001004', 'Multimedia Design', '2021-01-20',
'Active', 3),
('Rania Abu Ali', 'rania.abuali@ttc.edu', '0597001005', 'Business Applications', '2022-08-
25', 'Active', 4),
('Fadi Younis', 'fadi.younis@ttc.edu', '0597001006', 'Cybersecurity', '2023-03-12',
'Inactive', 2);

UPDATE Department SET CoordinatorInstructorID = 1 WHERE DepartmentID = 1;
UPDATE Department SET CoordinatorInstructorID = 3 WHERE DepartmentID = 2;
UPDATE Department SET CoordinatorInstructorID = 4 WHERE DepartmentID = 3;
UPDATE Department SET CoordinatorInstructorID = 5 WHERE DepartmentID = 4;

INSERT INTO Course (CourseCode, Title, Description, TrainingHours, Level, Status,
DepartmentID) VALUES
('CS101', 'Programming Fundamentals', 'Basic programming concepts using structured problem
solving.', 40, 'Beginner', 'Active', 1),
('DB201', 'Database Management Systems', 'Relational database design, SQL, and database
administration basics.', 45, 'Intermediate', 'Active', 1),
('WD301', 'Full Stack Web Development', 'Client-side and server-side web application
development.', 60, 'Advanced', 'Active', 1),
('MB302', 'Flutter Mobile Development', 'Mobile application development using Flutter.',
55, 'Advanced', 'Active', 1),
('NW101', 'Networking Essentials', 'Fundamentals of networks, devices, and network
services.', 35, 'Beginner', 'Active', 2),
('MM201', 'Digital Media Production', 'Digital image, audio, and video production
techniques.', 42, 'Intermediate', 'Active', 3),
('BA101', 'Business Productivity Tools', 'Practical use of common tools for business
work.', 30, 'Beginner', 'Active', 4),
('AI401', 'Applied Artificial Intelligence', 'Introduction to applied AI methods for
technical projects.', 70, 'Advanced', 'Active', 1);

INSERT INTO Room (RoomCode, RoomType, Capacity, Location) VALUES
('LAB-A', 'Computer Laboratory', 24, 'Building A, First Floor'),
('LAB-B', 'Computer Laboratory', 30, 'Building A, Second Floor'),
```

```
('ROOM-101', 'Regular Classroom', 35, 'Building B, First Floor'),
('MEDIA-1', 'Multimedia Laboratory', 20, 'Building C, Ground Floor');

INSERT INTO CourseOffering (CourseCode, Term, StartDate, EndDate, MaxTrainees, Status,
InstructorID, RoomID) VALUES
('CS101', 'Spring 2026', '2026-02-01', '2026-03-15', 24, 'Completed', 2, 1),
('DB201', 'Spring 2026', '2026-02-05', '2026-03-25', 24, 'Completed', 1, 2),
('WD301', 'Summer 2026', '2026-06-01', '2026-08-15', 20, 'In Progress', 2, 1),
('MB302', 'Summer 2026', '2026-06-10', '2026-08-20', 20, 'Open', NULL, 2),
('NW101', 'Autumn 2026', '2026-09-01', '2026-10-15', 30, 'Open', 3, 3),
('MM201', 'Autumn 2026', '2026-09-05', '2026-10-25', 20, 'Planned', 4, NULL),
('DB201', 'Winter 2027', '2027-01-10', '2027-03-01', 24, 'Planned', NULL, NULL),
('AI401', 'Spring 2027', '2027-03-15', '2027-05-30', 18, 'Planned', NULL, NULL);

INSERT INTO Trainee (FullName, Gender, DateOfBirth, Email, Phone, Qualification,
RegistrationDate) VALUES
('Mohammad Saleem', 'Male', '2003-04-12', 'mohammad@gmail.com', '0598111111', 'High
School', '2026-08-03'),
('Sara Mahmoud', 'Female', '2002-11-21', 'sara.mahmoud@gmail.com', '0598222222', 'Diploma
in IT', '2026-08-05'),
('Ahmad Darwish', 'Male', '2001-07-09', 'ahmad.darwish@gmail.com', '0598333333', 'High
School', '2026-08-11'),
('Hala Qasem', 'Female', '2004-02-18', 'hala.qasem@gmail.com', '0598444444', 'Bachelor of
Business', '2026-08-18'),
('Nour Odeh', 'Female', '2003-09-30', 'nour.odeh@gmail.com', '0598555555', 'Diploma in
Design', '2026-08-22'),
('Yousef Hamdan', 'Male', '2000-12-02', 'yousef.hamdan@gmail.com', '0598666666', 'Bachelor
of Computer Science', '2026-07-15'),
('Dima Taha', 'Female', '2002-05-27', 'dima.taha@gmail.com', '0598777777', 'High School',
'2026-08-29'),
('Khaled Mansour', 'Male', '2001-10-10', 'khaled.mansour@gmail.com', '0598888888',
'Diploma in Networking', '2026-09-02');

INSERT INTO Assessment (OfferingID, Title, AssessmentType, AssessmentDate, MaxMark,
Weight) VALUES
(1, 'Programming Quiz', 'Quiz', '2026-02-15', 20, 20),
(1, 'Programming Final', 'Final Examination', '2026-03-12', 60, 60),
(2, 'SQL Practical', 'Practical Assignment', '2026-02-28', 40, 40),
(2, 'Database Final', 'Final Examination', '2026-03-22', 60, 60),
(3, 'Web Project', 'Project', '2026-08-05', 100, 50),
(5, 'Networking Quiz', 'Quiz', '2026-09-20', 30, 30);

INSERT INTO CoursePrerequisite (CourseCode, PrerequisiteCourseCode) VALUES
('DB201', 'CS101'),
('WD301', 'CS101'),
('MB302', 'CS101'),
('AI401', 'DB201');

INSERT INTO Enrollment (TraineeID, OfferingID, EnrollmentDate, Status) VALUES
(1, 1, '2026-01-15', 'Completed'),
(2, 1, '2026-01-16', 'Completed'),
(3, 1, '2026-01-18', 'Completed'),
(1, 2, '2026-01-20', 'Completed'),
(2, 2, '2026-01-21', 'Completed'),
(4, 2, '2026-01-23', 'Failed'),
(1, 3, '2026-05-20', 'Registered'),
(2, 3, '2026-05-21', 'Registered'),
(5, 4, '2026-06-01', 'Registered'),
(6, 5, '2026-08-20', 'Registered'),
(7, 5, '2026-08-21', 'Registered'),
(8, 5, '2026-08-22', 'Registered');
```

```
INSERT INTO AssessmentResult (AssessmentID, TraineeID, Mark) VALUES
(1, 1, 17), (1, 2, 15), (1, 3, 12),
(2, 1, 52), (2, 2, 48), (2, 3, 39),
(3, 1, 35), (3, 2, 31), (3, 4, 22),
(4, 1, 54), (4, 2, 46), (4, 4, 30),
(5, 1, 88), (5, 2, 76);
GO
```

This is the link to the SSMS program for the DML file.

https://drive.google.com/file/d/1F-71Y9bJ4IhQxLfilB_RDgeARRsosBXd/view?usp=sharing

11. Validation and Constraint Testing

The following tests demonstrate that valid data can be inserted and that important invalid or duplicate operations are rejected by SQL Server. The failed tests are expected to produce SQL Server error messages. The actual messages and screenshots must be inserted after executing the statements in SQL Server.

This is the link to the SSMS program for the Validation and Constraint Testing file.

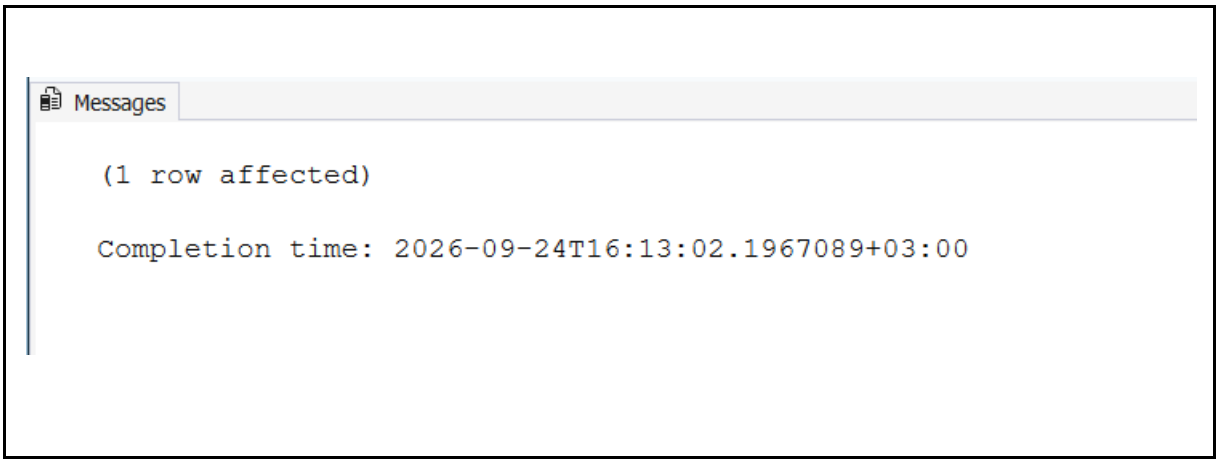
https://drive.google.com/file/d/1EPPtkyb_abZX0CIfrPhHvByirqLOiwZ-/view?usp=sharing

Test 1 – Valid Insert

Purpose: Verify that valid data can be inserted successfully.

Expected result: The insertion should succeed.

```
INSERT INTO Trainee (FullName, Gender, DateOfBirth, Email, Phone, Qualification,
RegistrationDate) VALUES ('Validation Test', 'Male', '2005-05-10', 'validation@test.com',
'0599000000', 'High School', '2026-09-24');
```

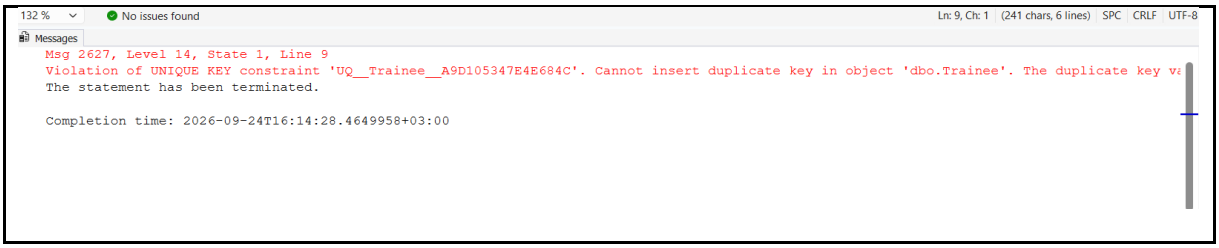


Test 2 – Duplicate Email

Purpose: Test the UNIQUE constraint on Trainee.Email.

Expected result: The insertion should fail because the email already exists.

```
INSERT INTO Trainee (FullName, Gender, DateOfBirth, Email, Phone, Qualification,
RegistrationDate) VALUES ('Duplicate Email Test', 'Male', '2004-01-01',
'mohammad@gmail.com', '0598111111', 'High School', '2026-09-01');
```

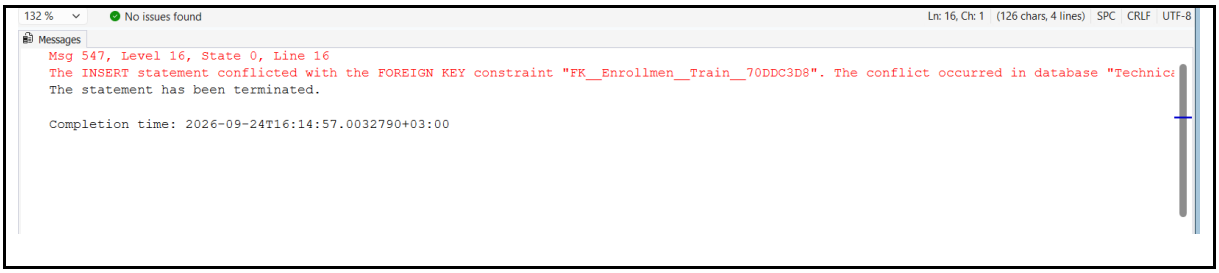


Test 3 – Invalid Foreign Key

Purpose: Test referential integrity using TraineeID 999.

Expected result: The insertion should fail because TraineeID 999 does not exist.

```
INSERT INTO Enrollment (TraineeID, OfferingID, EnrollmentDate, Status) VALUES (999, 1,
'2026-09-24', 'Registered');
```



Test 4 – NOT NULL Constraint

Purpose: Verify that FullName cannot be NULL.

Expected result: The insertion should fail because FullName is required.

```
INSERT INTO Trainee (FullName, Gender, DateOfBirth, Email, Phone, Qualification,
RegistrationDate) VALUES (NULL, 'Male', '2004-01-01', 'nulltest@test.com', '0598111111',
'High School', '2026-09-24');
```

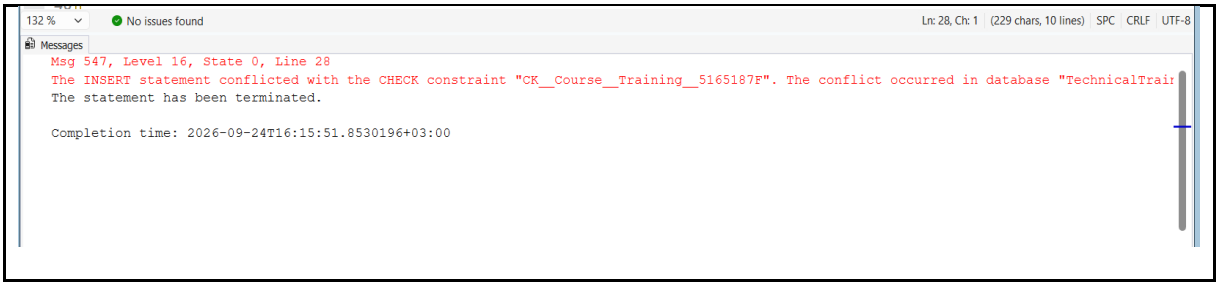


Test 5 – CHECK Constraint

Purpose: Verify that TrainingHours must be greater than zero.

Expected result: The insertion should fail because -10 violates the check constraint.

```
INSERT INTO Course (CourseCode, Title, Description, TrainingHours, Level, Status,
DepartmentID) VALUES ('TEST999', 'Invalid Course', 'Validation Test', -10, 'Beginner',
'Active', 1);
```

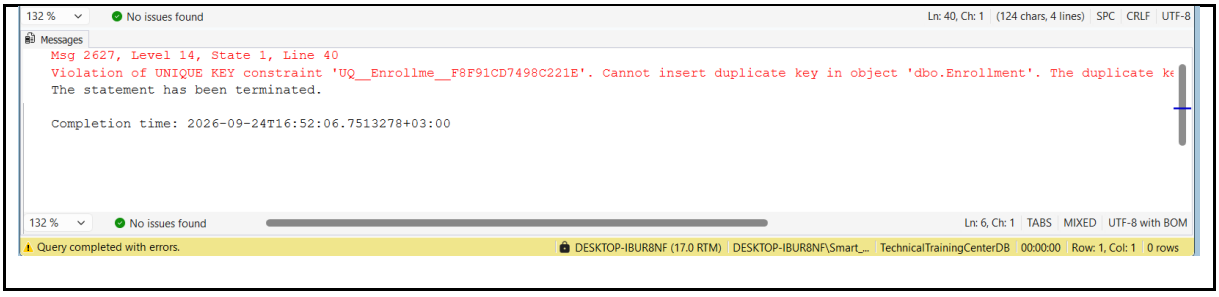


Test 6 – Duplicate Enrollment

Purpose: Verify the unique (TraineeID, OfferingID) rule.

Expected result: The insertion should fail because the trainee is already enrolled in Offering 1.

```
INSERT INTO Enrollment (TraineeID, OfferingID, EnrollmentDate, Status) VALUES (1, 1,
'2026-09-24', 'Registered');
```

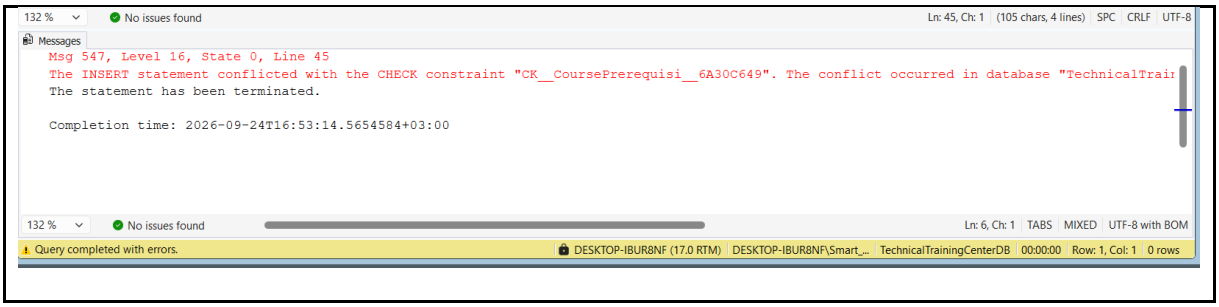


Test 7 – Self Prerequisite

Purpose: Verify that a course cannot be its own prerequisite.

Expected result: The insertion should fail because CourseCode must differ from PrerequisiteCourseCode.

```
INSERT INTO CoursePrerequisite (CourseCode, PrerequisiteCourseCode) VALUES ('CS101',
'CS101');
```

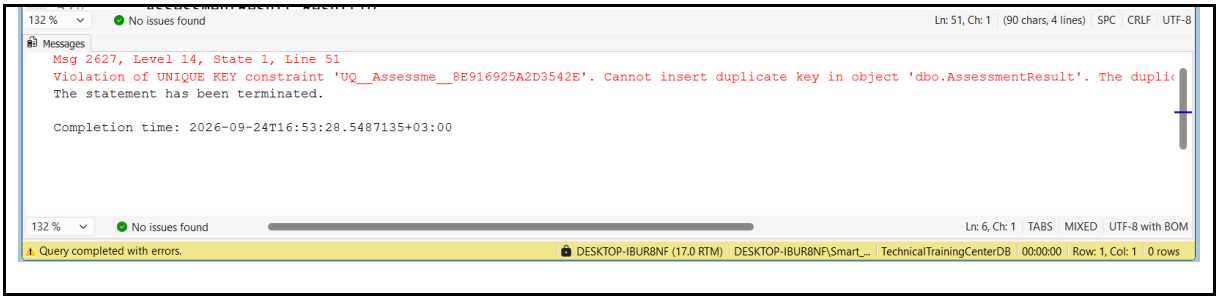


Test 8 – Negative Assessment Mark

Purpose: Verify that AssessmentResult.Mark cannot be negative.

Expected result: The insertion should fail because -5 violates the mark check constraint.

```
INSERT INTO AssessmentResult (AssessmentID, TraineeID, Mark) VALUES (1, 1, -5);
```



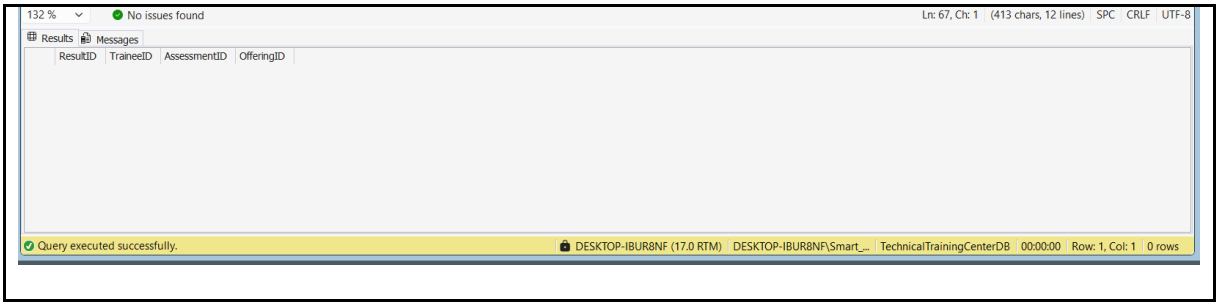
Business Rule Validation

The first query checks whether any stored mark is greater than the maximum mark of its assessment. The expected result is no rows. The second query checks whether any result belongs to a trainee who is not enrolled in the corresponding offering. The expected result is also no rows.

```
SELECT ar.ResultID, ar.AssessmentID, ar.TraineeID, ar.Mark, a.MaxMark
FROM AssessmentResult AS ar
INNER JOIN Assessment AS a ON ar.AssessmentID = a.AssessmentID
WHERE ar.Mark > a.MaxMark;
```



```
SELECT ar.ResultID, ar.TraineeID, a.AssessmentID, a.OfferingID
FROM AssessmentResult AS ar
INNER JOIN Assessment AS a ON ar.AssessmentID = a.AssessmentID
LEFT JOIN Enrollment AS e
    ON ar.TraineeID = e.TraineeID AND a.OfferingID = e.OfferingID
WHERE e.EnrollmentID IS NULL;
```



12. Required SQL Queries

The following portfolio contains all sixteen required query tasks from the assignment. Each query uses the finalized schema. Actual Result is intentionally left for insertion from SQL Server; no fake row counts or screenshots are included.

This is the link to the SSMS program for the Required SQL Query Portfolio file.

https://drive.google.com/file/d/1nHMl7dSehALie5mTgjHDR_GZOuzuHY0U/view?usp=sharing

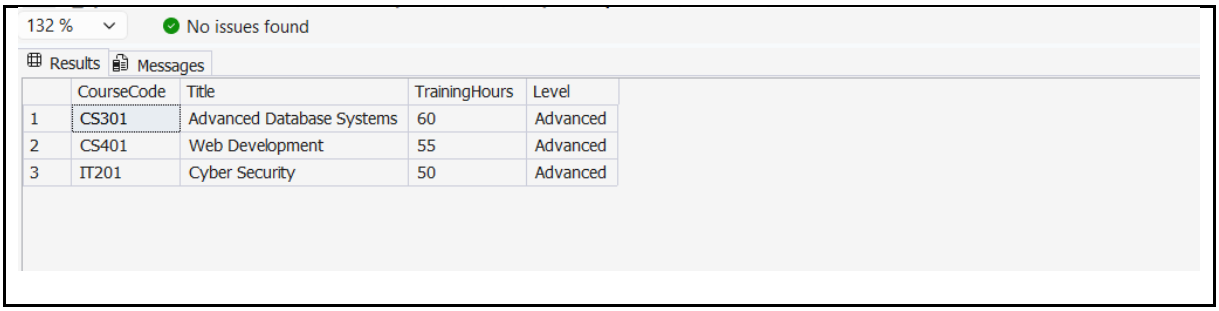
Q1. List all active Advanced courses, ordered by TrainingHours descending.

Purpose: Filtering and sorting.

SQL Query:

```
SELECT CourseCode, Title, TrainingHours, Level
FROM Course
WHERE Level = 'Advanced' AND Status = 'Active'
ORDER BY TrainingHours DESC;
```

Actual Result: To be inserted from SQL Server.



	CourseCode	Title	TrainingHours	Level
1	CS301	Advanced Database Systems	60	Advanced
2	CS401	Web Development	55	Advanced
3	IT201	Cyber Security	50	Advanced

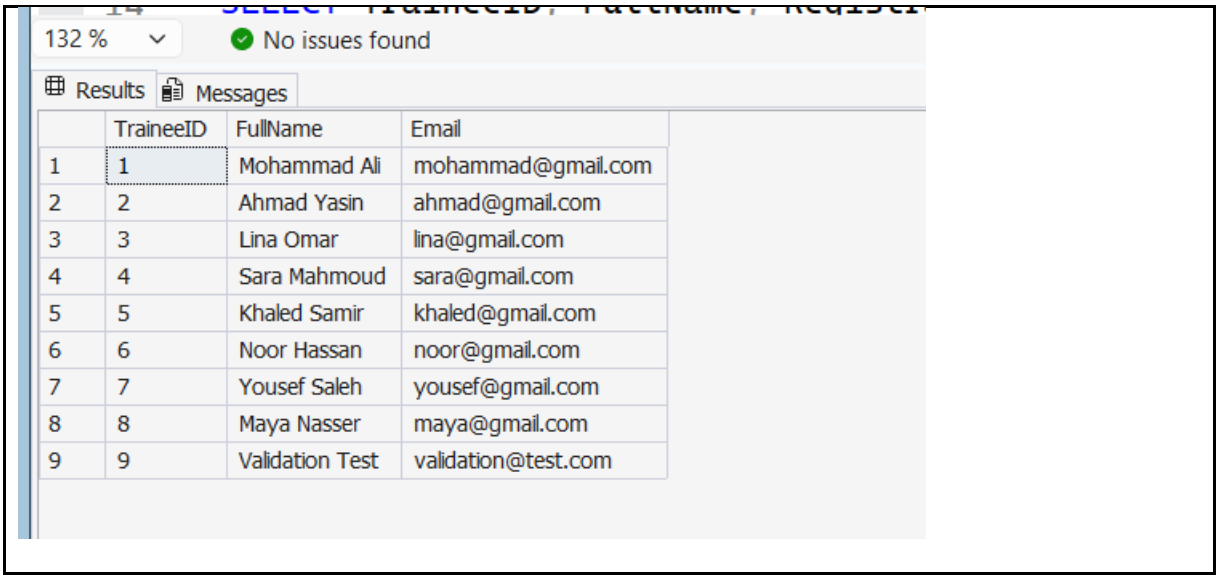
Q2. List trainees whose names contain the letter 'a'.

Purpose: Pattern matching.

SQL Query:

```
SELECT TraineeID, FullName, Email
FROM Trainee
WHERE FullName LIKE '%a%';
```

Actual Result: To be inserted from SQL Server.



	TraineeID	FullName	Email
1	1	Mohammad Ali	mohammad@gmail.com
2	2	Ahmad Yasin	ahmad@gmail.com
3	3	Lina Omar	lina@gmail.com
4	4	Sara Mahmoud	sara@gmail.com
5	5	Khaled Samir	khaled@gmail.com
6	6	Noor Hassan	noor@gmail.com
7	7	Yousef Saleh	yousef@gmail.com
8	8	Maya Nasser	maya@gmail.com
9	9	Validation Test	validation@test.com

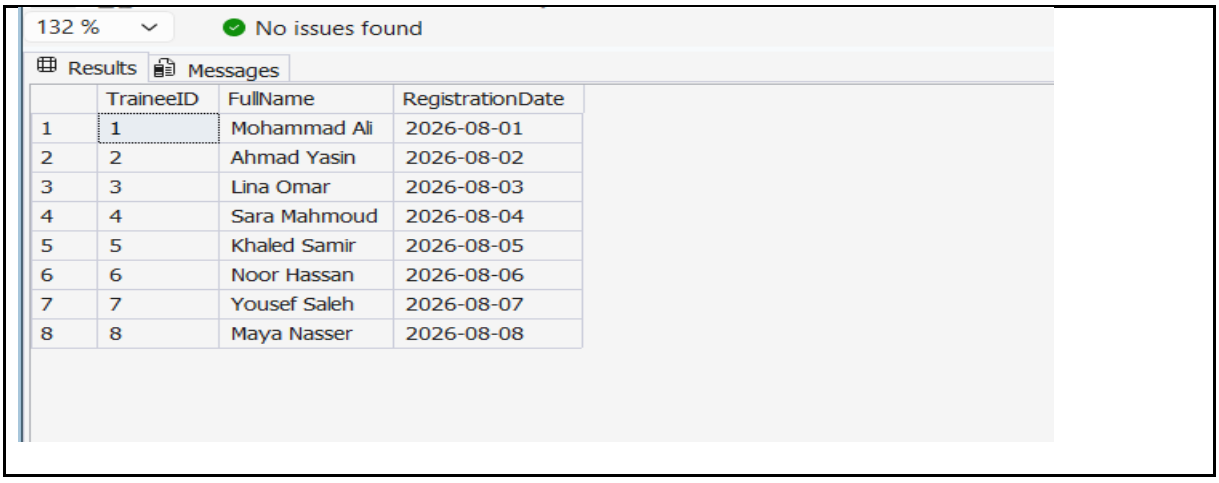
Q3. List trainees who registered during August 2026.

Purpose: Date range.

SQL Query:

```
SELECT TraineeID, FullName, RegistrationDate
FROM Trainee
WHERE RegistrationDate BETWEEN '2026-08-01' AND '2026-08-31'
ORDER BY RegistrationDate;
```

Actual Result: To be inserted from SQL Server.



	TraineeID	FullName	RegistrationDate
1	1	Mohammad Ali	2026-08-01
2	2	Ahmad Yasin	2026-08-02
3	3	Lina Omar	2026-08-03
4	4	Sara Mahmoud	2026-08-04
5	5	Khaled Samir	2026-08-05
6	6	Noor Hassan	2026-08-06
7	7	Yousef Saleh	2026-08-07
8	8	Maya Nasser	2026-08-08

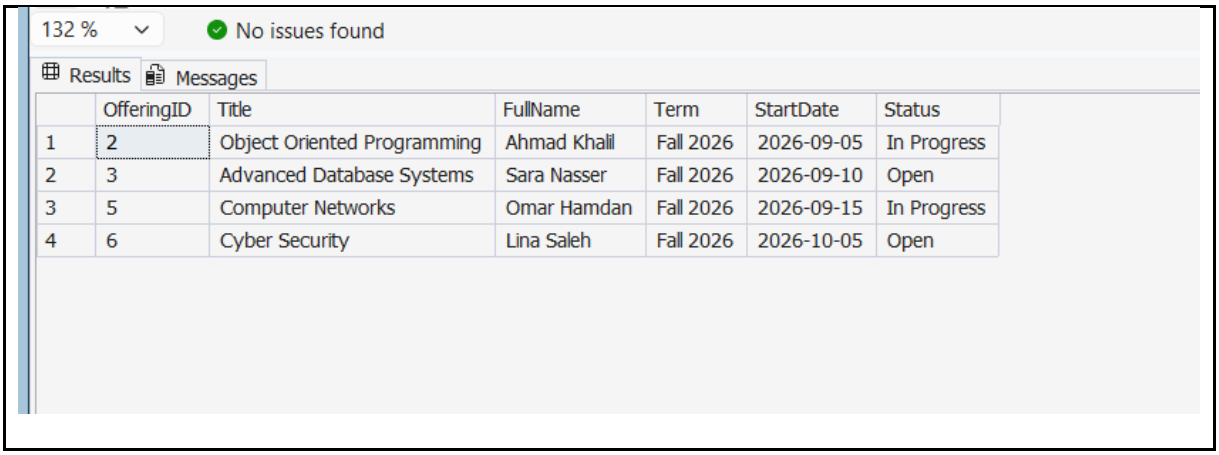
Q4. List active/open course offerings with course title and instructor name.

Purpose: Inner join.

SQL Query:

```
SELECT co.OfferingID, c.Title, i.FullName, co.Term, co.StartDate, co.Status
FROM CourseOffering AS co
INNER JOIN Course AS c ON co.CourseCode = c.CourseCode
LEFT JOIN Instructor AS i ON co.InstructorID = i.InstructorID
WHERE co.Status IN ('Open', 'In Progress');
```

Actual Result: To be inserted from SQL Server.



	OfferingID	Title	FullName	Term	StartDate	Status
1	2	Object Oriented Programming	Ahmad Khalil	Fall 2026	2026-09-05	In Progress
2	3	Advanced Database Systems	Sara Nasser	Fall 2026	2026-09-10	Open
3	5	Computer Networks	Omar Hamdan	Fall 2026	2026-09-15	In Progress
4	6	Cyber Security	Lina Saleh	Fall 2026	2026-10-05	Open

Q5. List each trainee with the courses and terms in which they are enrolled.

Purpose: Multi-table join.

SQL Query:

```
SELECT t.FullName AS TraineeName, c.Title AS CourseTitle,
       co.Term, e.Status
```

```
FROM Enrollment AS e
INNER JOIN Trainee AS t ON e.TraineeID = t.TraineeID
INNER JOIN CourseOffering AS co ON e.OfferingID = co.OfferingID
INNER JOIN Course AS c ON co.CourseCode = c.CourseCode;
```

Actual Result: To be inserted from SQL Server.



132 % | No issues found | Ln: 47, Ch: 54 | (372 chars, 12 lines) | SPC | CRLF | UTF-8

Results | Messages

	TraineeName	CourseTitle	Term	Status
1	Mohammad Ali	Programming Fundamentals	Fall 2026	Completed
2	Ahmad Yasin	Programming Fundamentals	Fall 2026	Completed
3	Lina Omar	Programming Fundamentals	Fall 2026	Completed
4	Mohammad Ali	Object Oriented Programming	Fall 2026	Registered
5	Ahmad Yasin	Object Oriented Programming	Fall 2026	Registered
6	Sara Mahmoud	Object Oriented Programming	Fall 2026	Registered
7	Mohammad Ali	Advanced Database Systems	Fall 2026	Registered
8	Khaled Samir	Advanced Database Systems	Fall 2026	Registered
9	Ahmad Yasin	Computer Networks	Fall 2026	Registered
10	Noor Hassan	Computer Networks	Fall 2026	Registered
11	Lina Omar	Cyber Security	Fall 2026	Registered
12	Yousef Saleh	Cyber Security	Fall 2026	Registered
13	Lina Omar	Graphic Design	Fall 2026	Completed
14	Sara Mahmoud	Graphic Design	Fall 2026	Completed

Query executed successfully. | DESKTOP-IBUR8NF (17.0 RTM) | DESKTOP-IBUR8NF\Smart_... | TechnicalTrainingCenterDB | 00:00:00 | Row: 1, Col: 1 | 14 rows

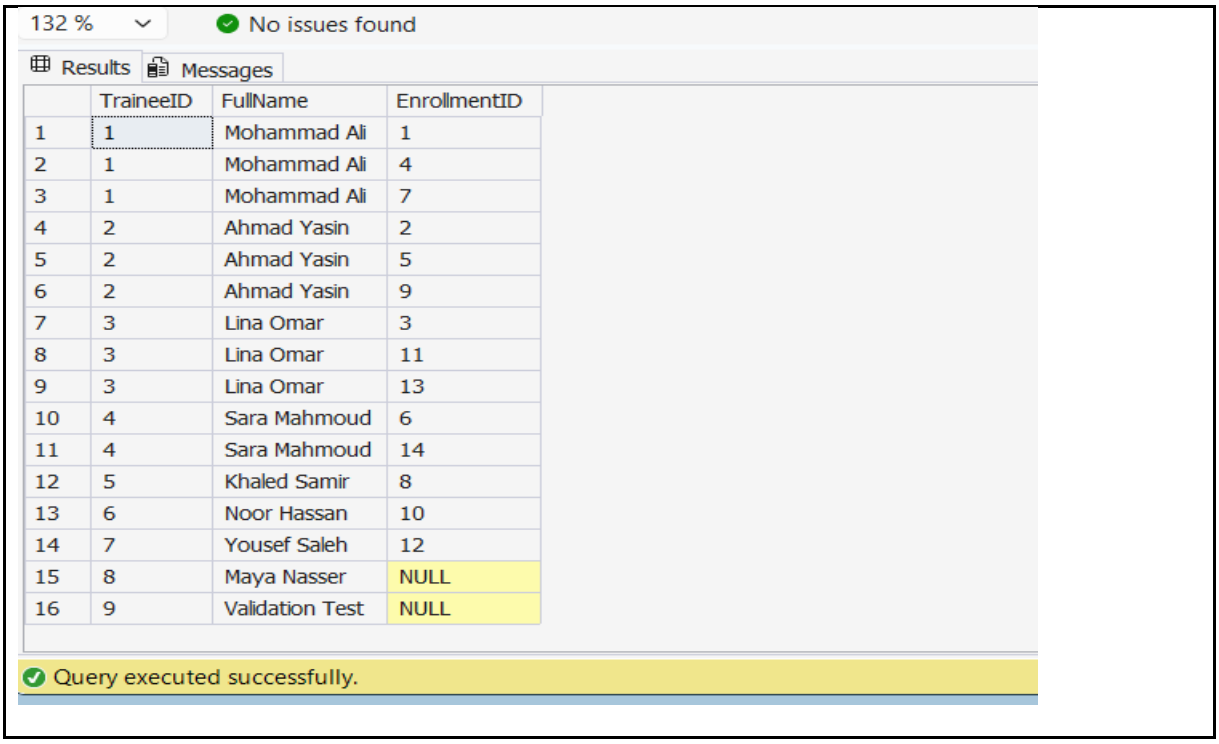
Q6. List all trainees, including trainees who have never enrolled in a course.

Purpose: Outer join.

SQL Query:

```
SELECT t.TraineeID, t.FullName, e.EnrollmentID
FROM Trainee AS t
LEFT JOIN Enrollment AS e ON t.TraineeID = e.TraineeID
ORDER BY t.TraineeID;
```

Actual Result: To be inserted from SQL Server.



132 % | No issues found

Results | Messages

	TraineeID	FullName	EnrollmentID
1	1	Mohammad Ali	1
2	1	Mohammad Ali	4
3	1	Mohammad Ali	7
4	2	Ahmad Yasin	2
5	2	Ahmad Yasin	5
6	2	Ahmad Yasin	9
7	3	Lina Omar	3
8	3	Lina Omar	11
9	3	Lina Omar	13
10	4	Sara Mahmoud	6
11	4	Sara Mahmoud	14
12	5	Khaled Samir	8
13	6	Noor Hassan	10
14	7	Yousef Saleh	12
15	8	Maya Nasser	NULL
16	9	Validation Test	NULL

Query executed successfully.

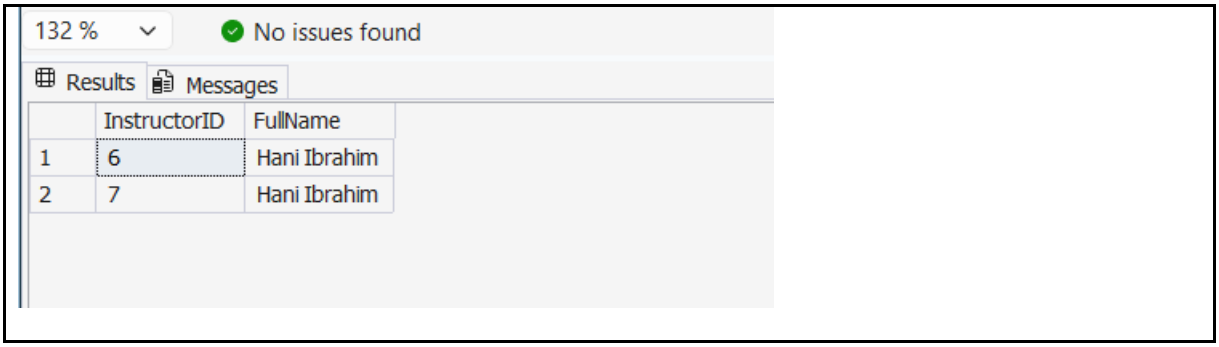
Q7. List instructors who have no current course offering assignment.

Purpose: Unmatched records.

SQL Query:

```
SELECT i.InstructorID, i.FullName
FROM Instructor AS i
LEFT JOIN CourseOffering AS co ON i.InstructorID = co.InstructorID
WHERE co.OfferingID IS NULL;
```

Actual Result: To be inserted from SQL Server.



	InstructorID	FullName
1	6	Hani Ibrahim
2	7	Hani Ibrahim

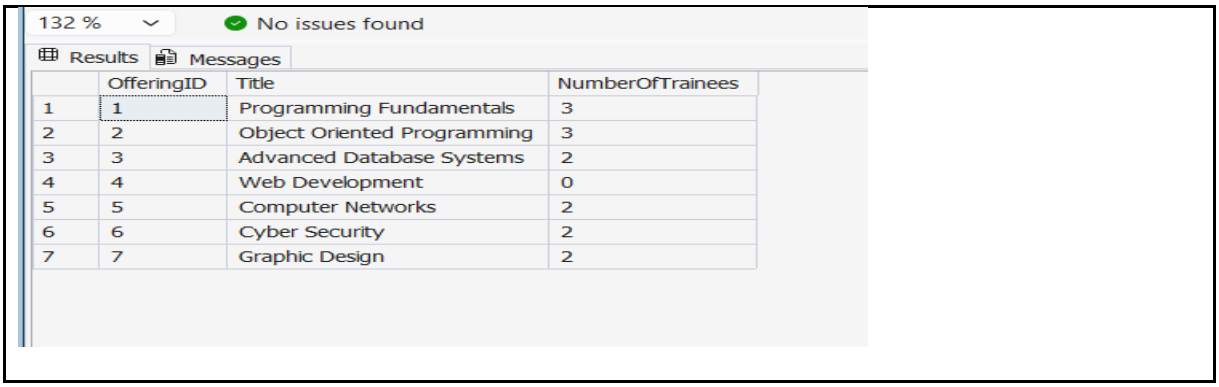
Q8. Count trainees in each course offering, including offerings with zero trainees.

Purpose: Aggregate and grouping.

SQL Query:

```
SELECT co.OfferingID, c.Title,
       COUNT(e.EnrollmentID) AS NumberOfTrainees
FROM CourseOffering AS co
INNER JOIN Course AS c ON co.CourseCode = c.CourseCode
LEFT JOIN Enrollment AS e ON co.OfferingID = e.OfferingID
GROUP BY co.OfferingID, c.Title;
```

Actual Result: To be inserted from SQL Server.



	OfferingID	Title	NumberOfTrainees
1	1	Programming Fundamentals	3
2	2	Object Oriented Programming	3
3	3	Advanced Database Systems	2
4	4	Web Development	0
5	5	Computer Networks	2
6	6	Cyber Security	2
7	7	Graphic Design	2

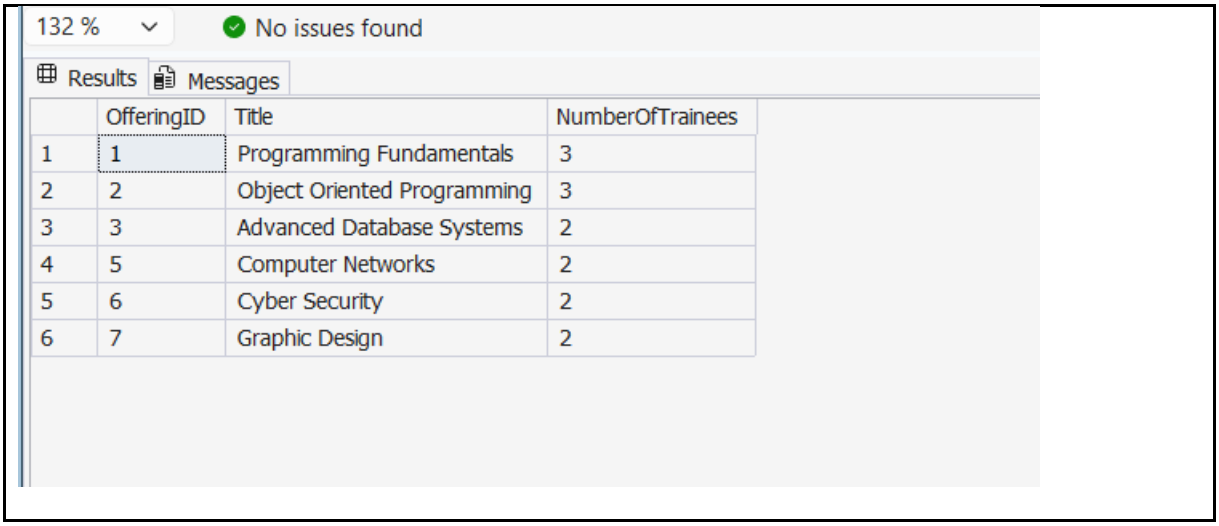
Q9. List course offerings with more than one trainee.

Purpose: HAVING.

SQL Query:

```
SELECT co.OfferingID, c.Title,
       COUNT(e.EnrollmentID) AS NumberOfTrainees
FROM CourseOffering AS co
INNER JOIN Course AS c ON co.CourseCode = c.CourseCode
LEFT JOIN Enrollment AS e ON co.OfferingID = e.OfferingID
GROUP BY co.OfferingID, c.Title
HAVING COUNT(e.EnrollmentID) > 1;
```

Actual Result: To be inserted from SQL Server.


132 % No issues found

Results Messages

	OfferingID	Title	NumberOfTrainees
1	1	Programming Fundamentals	3
2	2	Object Oriented Programming	3
3	3	Advanced Database Systems	2
4	5	Computer Networks	2
5	6	Cyber Security	2
6	7	Graphic Design	2

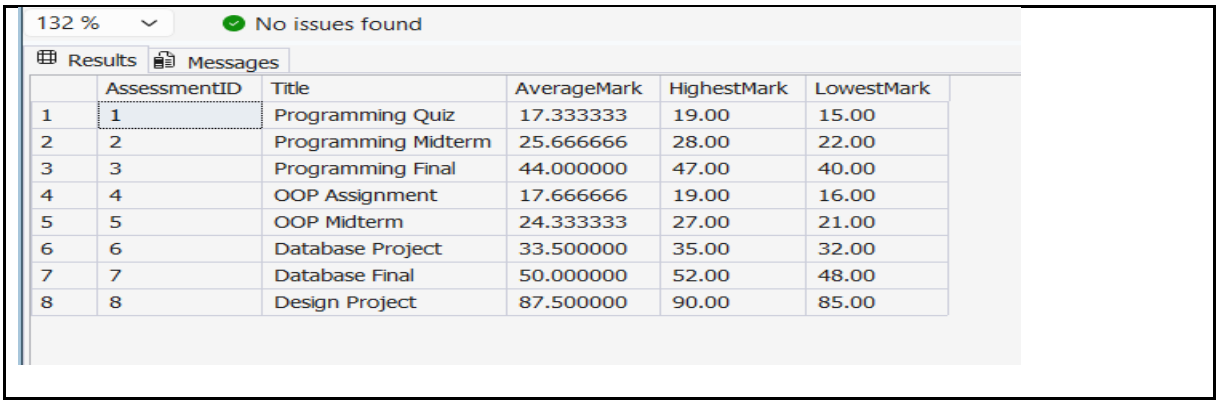
Q10. For each assessment, display the average, highest, and lowest mark.

Purpose: Aggregate analysis.

SQL Query:

```
SELECT a.AssessmentID, a.Title,
       AVG(ar.Mark) AS AverageMark,
       MAX(ar.Mark) AS HighestMark,
       MIN(ar.Mark) AS LowestMark
FROM Assessment AS a
INNER JOIN AssessmentResult AS ar ON a.AssessmentID = ar.AssessmentID
GROUP BY a.AssessmentID, a.Title;
```

Actual Result: To be inserted from SQL Server.



132 % No issues found

Results Messages

	AssessmentID	Title	AverageMark	HighestMark	LowestMark
1	1	Programming Quiz	17.333333	19.00	15.00
2	2	Programming Midterm	25.666666	28.00	22.00
3	3	Programming Final	44.000000	47.00	40.00
4	4	OOP Assignment	17.666666	19.00	16.00
5	5	OOP Midterm	24.333333	27.00	21.00
6	6	Database Project	33.500000	35.00	32.00
7	7	Database Final	50.000000	52.00	48.00
8	8	Design Project	87.500000	90.00	85.00

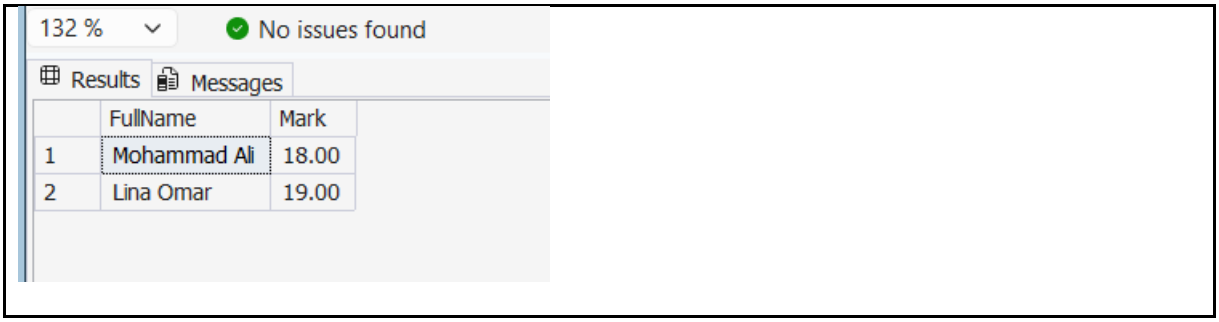
Q11. List trainees who scored above the average mark for AssessmentID 1.

Purpose: Subquery.

SQL Query:

```
SELECT t.FullName, ar.Mark
FROM AssessmentResult AS ar
INNER JOIN Trainee AS t ON ar.TraineeID = t.TraineeID
WHERE ar.AssessmentID = 1
  AND ar.Mark > (SELECT AVG(Mark)
                 FROM AssessmentResult
                 WHERE AssessmentID = 1);
```

Actual Result: To be inserted from SQL Server.



132 % No issues found

Results Messages

	FullName	Mark
1	Mohammad Ali	18.00
2	Lina Omar	19.00

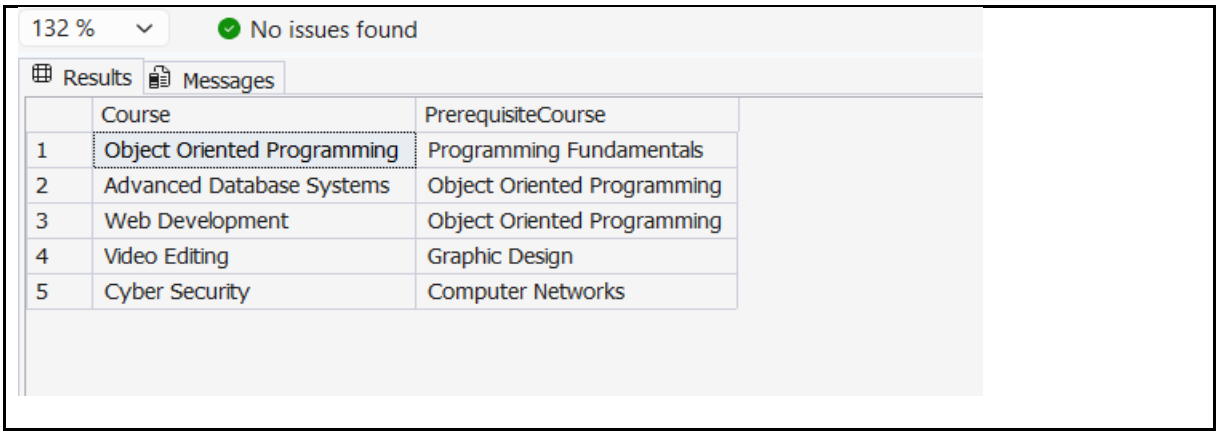
Q12. List each course together with its prerequisite course.

Purpose: Prerequisite query.

SQL Query:

```
SELECT c.Title AS Course, p.Title AS PrerequisiteCourse
FROM CoursePrerequisite AS cp
INNER JOIN Course AS c ON cp.CourseCode = c.CourseCode
INNER JOIN Course AS p ON cp.PrerequisiteCourseCode = p.CourseCode;
```

Actual Result: To be inserted from SQL Server.



	Course	PrerequisiteCourse
1	Object Oriented Programming	Programming Fundamentals
2	Advanced Database Systems	Object Oriented Programming
3	Web Development	Object Oriented Programming
4	Video Editing	Graphic Design
5	Cyber Security	Computer Networks

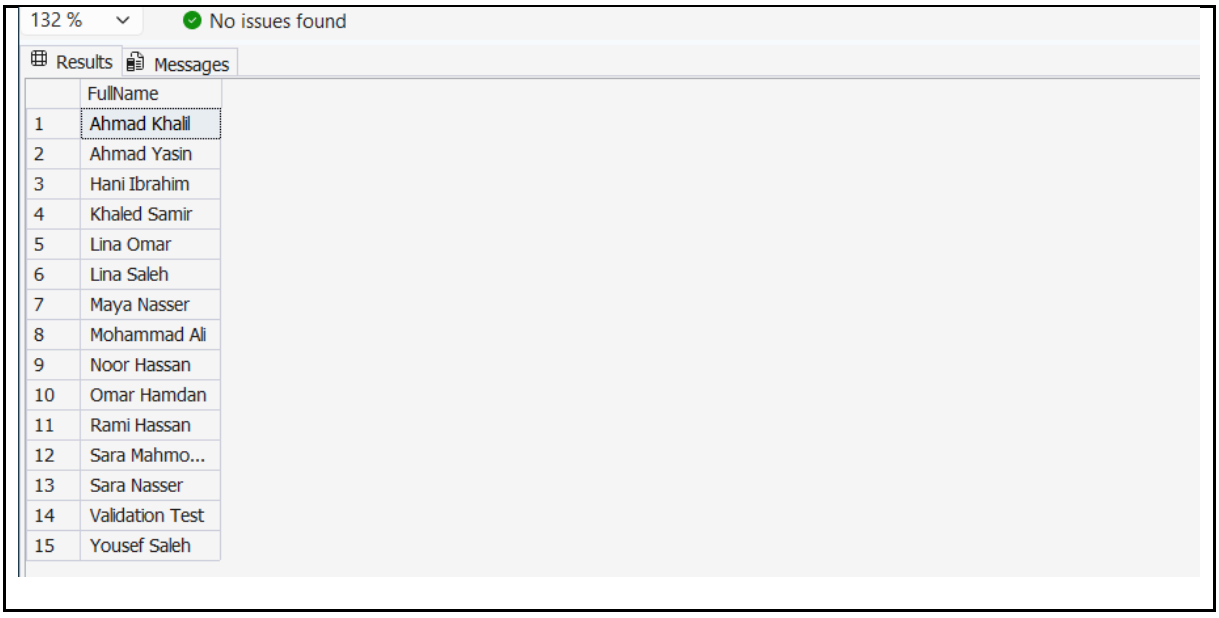
Q13. Display the names of all trainees and instructors using UNION.

Purpose: UNION.

SQL Query:

```
SELECT FullName FROM Trainee
UNION
SELECT FullName FROM Instructor;
```

Actual Result: To be inserted from SQL Server.



	FullName
1	Ahmad Khalil
2	Ahmad Yasin
3	Hani Ibrahim
4	Khaled Samir
5	Lina Omar
6	Lina Saleh
7	Maya Nasser
8	Mohammad Ali
9	Noor Hassan
10	Omar Hamdan
11	Rami Hassan
12	Sara Mahmo...
13	Sara Nasser
14	Validation Test
15	Yousef Saleh

Q14. Find trainees who are enrolled in both Offering 1 and Offering 2.

Purpose: INTERSECT.

SQL Query:

```
SELECT TraineeID FROM Enrollment WHERE OfferingID = 1
INTERSECT
SELECT TraineeID FROM Enrollment WHERE OfferingID = 2;
```

Actual Result: To be inserted from SQL Server.



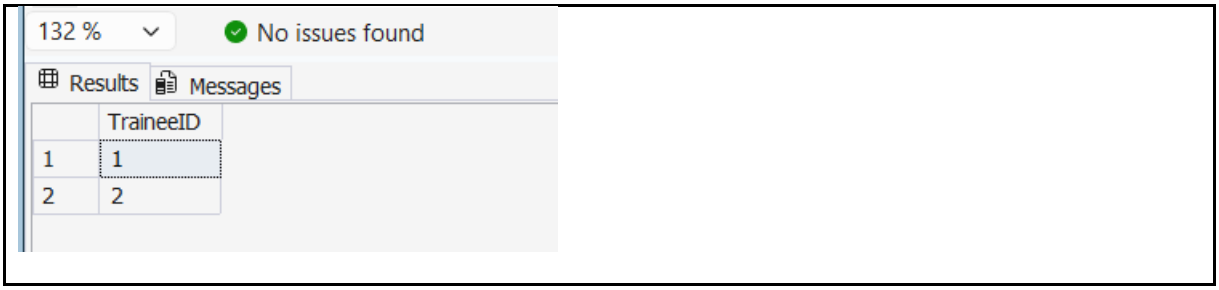
Q15. Find trainees who completed Offering 1 but did not complete Offering 7.

Purpose: EXCEPT.

SQL Query:

```
SELECT TraineeID FROM Enrollment
WHERE OfferingID = 1 AND Status = 'Completed'
EXCEPT
SELECT TraineeID FROM Enrollment
WHERE OfferingID = 7 AND Status = 'Completed';
```

Actual Result: To be inserted from SQL Server.



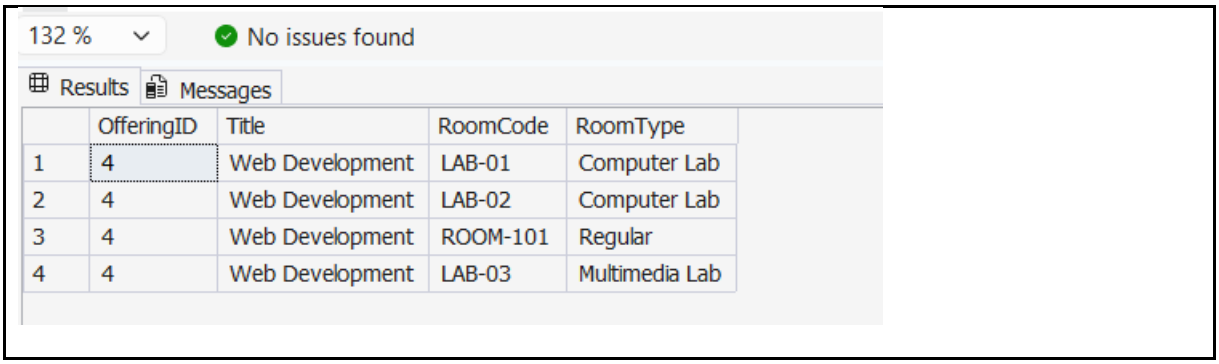
Q16. Show all combinations of planned course offerings and rooms.

Purpose: CROSS JOIN.

SQL Query:

```
SELECT co.OfferingID, c.Title, r.RoomCode, r.RoomType
FROM CourseOffering AS co
INNER JOIN Course AS c ON co.CourseCode = c.CourseCode
CROSS JOIN Room AS r
WHERE co.Status = 'Planned';
```

Actual Result: To be inserted from SQL Server.



	OfferingID	Title	RoomCode	RoomType
1	4	Web Development	LAB-01	Computer Lab
2	4	Web Development	LAB-02	Computer Lab
3	4	Web Development	ROOM-101	Regular
4	4	Web Development	LAB-03	Multimedia Lab

Explanation: The CROSS JOIN creates possible combinations between planned offerings and rooms. This is useful as an initial scheduling list because it shows candidate room choices before additional scheduling rules are applied.

13. Conclusion

This project resulted in a relational database for managing courses, course offerings, instructors, trainees, enrollments, assessments, assessment results, rooms, departments, and course prerequisites. The database was tested using constraints, realistic sample data, validation statements, and the required SQL query portfolio. The final design provides a clear path from the training center business requirements to a consistent SQL Server implementation.